

Пароль менеджері веб- қосымшасын әзірлеу

Орындаған: Асетова Анель, Реймқұл Акерке

Тобы: ИС-24-08К

Тексерген: Сатыбалдиева Феруза

Тақырып және мақсат

Мәселе

Сайттар көп, парольдер көп. Адамдар оларды ұмытады немесе бәріне бір парольді қояды.

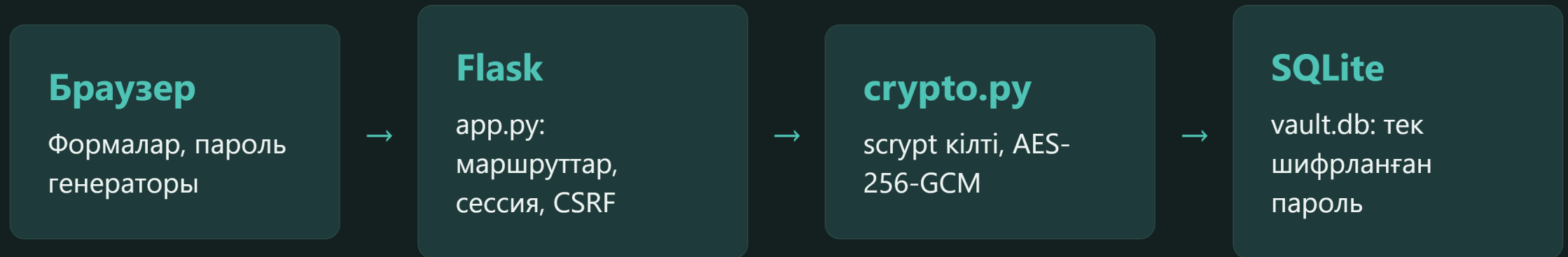
Шешім

Барлық парольді шифрланған сейфте сақтап, бір master парольмен ашу.

Нәтиже

Жұмыс істейтін веб-қосымша, коды GitHub-қа салынған.

Архитектура және қауіпсіздік моделі



Ашық пароль дерекқорға ешқашан түспейді. Шифрлау кілті master парольден алынады және сақталмайды.

Кодтау: жоба құрылымы

Файл	Міндеті
app.py	Маршруттар, сессия, CSRF қорғанысы
crypto.py	Кілт алу және шифрлау
templates/	HTML беттер: кіру, тізім, форма
static/	Стиль және JavaScript
vault.db	SQLite дерекқоры

Негізгі код 1: шифрлау (crypto.py)

```
def derive_key(master_password, salt):
    kdf = Scrypt(salt=salt, length=32, n=2**15, r=8, p=1)
    return kdf.derive(master_password.encode("utf-8"))

def encrypt(key, plaintext):
    nonce = os.urandom(12)
    ct = AESGCM(key).encrypt(nonce, plaintext.encode("utf-8"), None)
    return base64.b64encode(nonce + ct).decode("ascii")

def decrypt(key, token):
    raw = base64.b64decode(token)
    nonce, ct = raw[:12], raw[12:]
    return AESGCM(key).decrypt(nonce, ct, None).decode("utf-8")
```

- **derive_key** master парольден 256 бит кілт жасайды. Scrypt әдейі баяу әрі жадты көп қолданады, сондықтан парольді болжау қиын
- **salt** әр пайдаланушыға жеке кездейсоқ мән: бірдей парольдің кілті әртүрлі болады
- **encrypt** AES-256-GCM қолданады, әр шифрлауда жаңа nonce
- GCM деректің өзгергенін анықтайды: бүлінсе, decrypt қате береді

Негізгі код 2: master парольді хэштеу

```
ph = PasswordHasher() # Argon2

INSERT INTO users (username, pw_hash, salt)
VALUES (?, ?, ?)
→ (username, ph.hash(password), crypto.new_salt())
```

- Master пароль дерекқорда ашық күйінде сақталмайды, тек Argon2 хэші сақталады
- Хэшті кері қайтарып, парольді алу мүмкін емес: кіргенде енгізілген пароль хэшпен салыстырылады
- Дерекқор ұрланса да, master пароль көрінбейді

Негізгі код 3: кіру және кілтті жадта ұстау

```
ph.verify(user["pw_hash"], password)

sid = secrets.token_hex(32)
KEYS[sid] = crypto.derive_key(password, user["salt"])
session["sid"] = sid

# шыққанда:
KEYS.pop(sid)
```

- Пароль дұрыс болса, сол парольден шифрлау кілті жасалады
- Кілт KEYS сөздігінде, яғни сервер жадында тұрады
- Cookie-де тек кездейсоқ sid болады: cookie ұрланса да, кілт ұрланбайды
- Шыққанда кілт жадтан өшіріледі

Негізгі код 4: сақтау, ашу, CSRF

```
# сақтау
crypto.encrypt(current_key(), f["password"])

# ашу
crypto.decrypt(current_key(), row["password_enc"])

# CSRF қорғанысы
secrets.compare_digest(sent, session.get("csrf", ""))
```

- Сайт парольдері дерекқорға тек шифрланған күйде жазылады (password_enc)
- Ашу үшін master парольмен кіру керек, сонда ғана кілт жадта болады
- CSRF токени жалған сұраныстарды тоқтатады, токенсіз сұраныс 400 қатесін қайтарады

Функционалды және қауіпсіздік тестілеу

Тексеру	Нәтиже
Тіркелу және кіру	Жұмыс істейді
Пароль қосу, тізімде көру, ашу	Жұмыс істейді
Дерекқордағы пароль	Шифрланған, ашық мәтін жоқ
CSRF токенсіз сұраныс	Қабылданбайды (400)

Түзетулер: [тапқан қателеріңізді осында жазыңыз]

Қорытынды және даму жолдары

Не істелді

- Шифрланған сейф жұмыс істейді
- Argon2, scrypt, AES-256-GCM
- Код GitHub-та

Әрі қарай

- HTTPS және gunicorn
- Екі факторлы кіру
- Логинді де шифрлау

Үш қорғаныс қабаты: master пароль Argon2-мен хэштеледі, сайт парольдері AES-256-GCM-мен шифрланады, шифрлау кілті master парольден алынады және ешқайда сақталмайды. Дерекқор ұрланса да, парольдерді оқу мүмкін емес.